

Rapport d'avancement — PFE Détection de fraude fiscale par Graph ML

Projet pfe-fraude-graph-ml · Branche impl-pipeline · Date : 2026-06-11

1. Objectif du projet

Le projet vise à construire un outil de triage qui attribue un score de risque de fraude à des contribuables, entreprises ou particuliers. Ce score sert à prioriser les dossiers à examiner par un humain : il ne constitue pas une décision automatique de fraude.

La question de recherche centrale est : est-ce qu'exploiter les relations entre contribuables avec une approche graphe, notamment GraphSAGE, améliore la détection de fraude par rapport à une approche classique qui regarde chaque contribuable isolément, comme XGBoost ?

Le périmètre est volontairement limité à la fraude fiscale. Sont exclus : TVA spécifique, blanchiment, banque et cartes bancaires.

2. Idée scientifique

Un modèle classique de Machine Learning analyse surtout les caractéristiques individuelles d'un contribuable : chiffre d'affaires, retards, ratios, historique déclaratif, etc.

Le Graph Machine Learning ajoute une dimension relationnelle : même dirigeant, même adresse, même comptable, relation client-fournisseur, participation ou proximité avec des entités déjà connues comme frauduleuses.

L'hypothèse est qu'un contribuable peut paraître peu suspect individuellement, mais devenir prioritaire si son voisinage dans le graphe ressemble à celui de fraudeurs connus.

3. Choix d'architecture et justification

Modules Python comme source de vérité. La logique métier est placée dans **src/**, tandis que les notebooks Colab restent des wrappers. Cela évite la duplication, rend le code testable et facilite la réutilisation.

GraphSAGE comme modèle principal. Il est simple, robuste, standard dans PyTorch Geometric et adapté au périmètre d'un PFE.

XGBoost comme baseline. Il représente une approche tabulaire forte, sans message-passing graphe, et permet de mesurer l'apport réel des relations.

Pas de base graphe. Le graphe est construit en mémoire avec PyTorch Geometric. Neo4j/ArangoDB ne sont pas nécessaires à ce stade.

4. Avancement réalisé

Cora. GraphSAGE atteint une accuracy test de 0.798, ce qui valide la mécanique PyTorch Geometric et la boucle d'entraînement.

YelpChi. Une pipeline fraude complète a été construite : chargement, normalisation, GraphSAGE, XGBoost, XGBoost+graph, métriques, benchmark multi-seed et analyse relationnelle.

Fiscal. Un stub **src/data/fiscal_graph.py** prépare la construction future d'un graphe fiscal hétérogène avec HeteroData.

5. Changement important : abandon de DGL

Le plan initial prévoyait de charger YelpChi avec DGL. En pratique, DGL a posé des problèmes de compatibilité, notamment avec graphbolt et torchdata.datapipes sur Colab.

Le choix final est de charger directement le fichier .mat YelpChi avec scipy. Ce choix est plus robuste, fonctionne localement et sur Colab, supprime une dépendance fragile et garde les mêmes données.

6. Structure du code

src/config.py : seed, device CPU/GPU et hyperparamètres d'entraînement.

src/data/ : chargement de Cora, YelpChi, préparation du futur graphe fiscal, features de graphe et normalisation.

src/models/ : GraphSAGE, GAT et GNN multi-relationnel.

src/train/ : entraînement des GNN et baseline XGBoost.

src/eval/metrics.py : AUC-ROC, AUC-PR, F1-macro, GMean et Recall@k.

src/experiments/ : benchmarks, sweep des relations et expérience multi-relationnelle.

tests/ : tests unitaires vérifiant les métriques, modèles, loaders et absence de fuite de labels.

docs/ : résultats, spécification, plan et ce rapport.

7. Résultats actuels

Cora : accuracy test de 0.798.

YelpChi — GraphSAGE (homo) : AUC-ROC 0.894 ± 0.002 ; AUC-PR 0.676 ± 0.006 .

YelpChi — XGBoost : AUC-ROC 0.946 ± 0.003 ; AUC-PR 0.819 ± 0.010 .

YelpChi — XGBoost+graph : AUC-ROC 0.951 ± 0.002 ; AUC-PR 0.830 ± 0.009 .

Conclusion provisoire : sur YelpChi avec la relation homo, XGBoost bat GraphSAGE. Mais cela s'explique par la faible qualité relationnelle du graphe homo pour le message-passing.

8. Analyse des relations

YelpChi contient plusieurs relations : homo, net_rur, net_rtr et net_rsr.

La relation la plus intéressante est net_rur, correspondant à R-U-R : deux avis reliés parce qu'ils sont écrits par le même utilisateur.

Homophilie observée : net_rur = 0.996 ; homo = 0.773 ; net_rsr = 0.772 ; net_rtr = 0.759.

net_rur est donc presque parfaitement homophile, mais elle est sparse : environ 48,1 % des nœuds sont isolés. Elle donne un signal très propre, mais ne couvre pas tout le graphe.

9. Pourquoi XGBoost bat GraphSAGE actuellement

GraphSAGE fonctionne bien si les voisins sont informatifs. Sur YelpChi, la relation homo est presque non homophile : les voisins ne partagent pas toujours le même label.

Dans ce cas, le message-passing peut lisser ou diluer le signal frauduleux. XGBoost, lui, exploite très bien les features tabulaires et n'est pas perturbé par un graphe bruité.

La bonne conclusion n'est donc pas que le Graph ML est inutile, mais que sa performance dépend fortement de la qualité des relations.

10. Pourquoi continuer en Graph ML

La relation `net_rur` montre que la structure du graphe porte bien une information : l'AUC-ROC de GraphSAGE augmente quand la relation est plus homophile.

Les prochaines étapes sont justifiées : GAT pour pondérer les voisins, GNN multi-relationnel pour combiner qualité et couverture, puis graphe fiscal hétérogène pour représenter les vraies entités métier.

11. Homogène, multi-relationnel et hétérogène

Actuellement, Cora et YelpChi sont utilisés surtout comme graphes homogènes : les nœuds sont du même type.

YelpChi est aussi multi-relationnel, car plusieurs types de relations existent entre les mêmes types de nœuds.

Pour la vraie IA fiscale, le choix le plus cohérent est un graphe hétérogène : entreprises, particuliers, dirigeants, adresses, comptables, déclarations et relations commerciales. Le fichier `fiscal_graph.py` prépare déjà cette direction avec HeteroData.

12. Métriques choisies

Les métriques utilisées sont AUC-ROC, AUC-PR, F1-macro, GMean et Recall@k.

L'accuracy n'est pas utilisée comme métrique principale, car la fraude est déséquilibrée. Un modèle qui prédit toujours non-fraudeur peut avoir une bonne accuracy tout en étant inutile.

Recall@k est particulièrement adapté au triage : il mesure combien de vrais fraudeurs sont retrouvés dans les dossiers les plus prioritaires.

13. État actuel et prochaines étapes

Le projet dispose d'une pipeline GNN fonctionnelle, d'une baseline XGBoost, d'une variante XGBoost+graph, de métriques adaptées, de tests automatisés, de résultats documentés et d'un stub fiscal hétérogène.

Prochaines étapes : benchmarker GAT, terminer le benchmark multi-relationnel, construire le vrai graphe fiscal lorsque les données arrivent et documenter les limites dans le mémoire.

14. Limites à mentionner

YelpChi est un banc d'essai public de faux avis, pas une preuve directe pour la fraude fiscale.

Dans le fiscal, seuls les contribuables déjà enquêtés ont un label ; les autres sont inconnus, pas forcément non-fraudeurs.

Le modèle fournit un score de risque pour prioriser les contrôles, pas une preuve de fraude.

PC-GNN et CARE-GNN sont cités comme références, mais non reproduits ; il ne faut pas comparer directement les chiffres si les protocoles diffèrent.

15. Conclusion provisoire

Le projet a atteint un stade solide : la pipeline Graph ML est fonctionnelle, testée et comparée à une baseline forte.

Les résultats actuels montrent que XGBoost bat GraphSAGE sur YelpChi avec la relation homo, mais l'analyse relationnelle montre que le graphe reste porteur d'information.

La suite logique est de passer vers des modèles plus adaptés aux relations multiples, puis vers un graphe fiscal hétérogène réel. C'est dans ce contexte métier que le Graph ML a le plus de chances d'apporter une valeur ajoutée.

Sources utilisées : FEUILLE_DE_ROUTE.md, AVANCEMENT.md, docs/RESULTATS.md, docs/superpowers/specs/, docs/superpowers/plans/, src/ et tests/.